

Little Gerry — Security, Encryption & Access Control

Last updated: 2026-07-06 · Applies to: v2.13.2 and later

This document describes how Little Gerry protects information: how data is encrypted, how it is transmitted, and who is authorized to access, edit, and view it. It is written to be read by both technical and non-technical stakeholders — the first section of each topic is plain-language, followed by the technical detail and the source of truth in code.

1. Security model at a glance

Little Gerry is a **single-user desktop application**. Each person runs their own private copy on their own computer:

- The backend (FastAPI/Python) runs **locally** and is bound to the loopback interface `127.0.0.1:8000` — it is **not** reachable from the network, the internet, or other machines.
- The frontend (React) runs inside a desktop window (pywebview / WebView2) on the same machine and talks only to that local backend.
- Application data lives in a **local PostgreSQL database** and in **per-user files** under the user's home directory. Nothing is stored on a shared server.

Because every install is private to one operating-system user account, the primary security boundary is **the user's own machine and OS login**. The controls below defend the data that lives there and govern what leaves the machine.

Concern	Mechanism
Who can sign in	Google SSO restricted to approved company domains
Proving identity per request	Signed JWT bearer tokens (HS256)
What a signed-in user may do	Role- and permission-based access control

Concern	Mechanism
Protecting stored secrets	Operating-system keyring (Credential Manager / Keychain)
Protecting stored files & credentials	Fernet authenticated encryption (AES-128 + HMAC)
Detecting tampering with history	SHA-256 hash-chained audit log
Trusting the installed app	Signed Windows installer + signed/notarized macOS package

2. Who is authorized (authentication)

2.1 Sign-in is Google SSO, domain-restricted

Sign-in is performed through **Google Workspace OAuth 2.0**. Only accounts on approved company domains may sign in; any other Google account is rejected before an application session is ever created.

- Approved domains: `pmi-llc.com` and `precisianmedical.com` (`_ALLOWED_DOMAINS` in `backend/routers/auth.py`).
- The Google **ID token** is decoded, the verified `email` claim is read, and the domain is checked against the allow-list. A non-approved domain raises an authorization error and no session is issued (`backend/routers/auth.py`).

2.2 Accounts are provisioned automatically on first sign-in

The first time an approved user signs in, an account is created for them automatically (`backend/routers/auth.py`):

- The application **owner** (`settings.admin_email`) is provisioned with the **admin** role.
- Everyone else is provisioned as a full-access **member**.
- Because sign-in is SSO-only, each account is given an **unusable random password** (`secrets.token_urlsafe(32)`, bcrypt-hashed) so the local password

login path can never be used to impersonate an SSO user.

- The event is recorded to the audit log as `user.auto_provisioned`.

2.3 Sessions and tokens

After sign-in the backend issues two JSON Web Tokens

([backend/services/auth/service.py](#)):

Token	Lifetime	Purpose
Access token	60 minutes	Sent on every API request to prove identity
Refresh token	30 days	Used to obtain a new access token without re-login

- Tokens are **signed with HS256** using a secret held in the OS keyring (see §5). Signature is verified on every request; an invalid or expired token is rejected with `401`.
- The access token carries the user id (`sub`), `role`, and a session id (`jti`).
- **Raw tokens are never stored.** The server persists only a **SHA-256 hash** of the refresh token in the `user_sessions` table ([backend/models/db/user.py](#), `_hash_token` in [service.py](#)). Sessions can be revoked (`revoked_at`) and expire (`expires_at`).
- On the client, tokens are held in the app's local web-view storage and sent as an `Authorization: Bearer <token>` header. For the streaming chat WebSocket, the access token is passed as a `?token=` query parameter and validated the same way ([backend/main.py](#)).

2.4 Local password login (fallback)

A traditional email + password login exists ([backend/routers/auth.py](#)) and is protected with **bcrypt** password hashing (per-hash random salt via `gensalt()`; `hash_password` / `verify_password` in [service.py](#)). In normal operation accounts are SSO-provisioned with random passwords, so this path is effectively unused, but it is held to the same hashing standard.

3. What a signed-in user may do (authorization / access control)

Every protected endpoint requires a valid access token via the `get_current_user` dependency ([backend/dependencies.py](#)). On top of authentication, Little Gerry enforces **role- and permission-based access control**.

3.1 Roles

Role	Who	Capabilities
<code>admin</code>	The application owner (<code>settings.admin_email</code>)	Full access, including admin-only operations and unconditional regulatory-file write
<code>member</code>	All other approved-domain users	Full day-to-day access to their own workspace

Admin-only routes are guarded by the `require_admin` dependency, which returns `403 Forbidden` for non-admins ([backend/dependencies.py](#)).

3.2 Fine-grained permission: regulatory files

The regulatory document store has a dedicated per-user permission flag, `can_write_regulatory` ([backend/models/db/user.py](#)):

- **Everyone can read** regulatory files.
- **Create / edit / rename / move / delete** requires the `require_regulatory_write` dependency to pass: the user must be an `admin` **or** have `can_write_regulatory = True`; otherwise the request is rejected with `403` ([backend/dependencies.py](#)).

3.3 Data isolation

Because each install is a single user's private copy, a user only ever sees the data in **their own** local database and file store. There is no cross-user data access surface — one person's install cannot read another person's data.

4. How information is protected at rest (encryption)

4.1 Master secrets live in the OS keyring— never on disk

All top-level secrets are stored in the **operating system's secure credential store** via the **keyring** library (service name **pmi-agent**), **never** in plaintext files, environment variables, or the database ([backend/config.py](#)):

- Windows → **Credential Manager**
- macOS → **Keychain**
- Linux → **Secret Service**

Secrets held there include:

- **JWT signing secret** — a 64-byte URL-safe random value generated on first run.
- **Fernet encryption key** — generated on first run (see below).
- **Cloud provider API keys** (e.g. the LLM / embedding provider keys).

4.2 Sensitive data files are encrypted with Fernet

Sensitive on-disk content is encrypted using **Fernet** from the **cryptography** library. Fernet provides **authenticated symmetric encryption**: **AES-128 in CBC mode** for confidentiality plus **HMAC-SHA256** for integrity/tamper detection, with the key sourced from the OS keyring (§4.1).

Fernet-encrypted data includes:

Data	Where	Source
Chat conversation attachments	Local file store	backend/services/chat_attachments.py
Knowledge Base documents	Local file store	backend/services/documents/ingestion.py
Odoo ERP API key	Database column	backend/services/odoo_service.py

Files are written encrypted and only decrypted in memory when the owning user requests them.

4.3 Database

Application records live in a **local PostgreSQL** database on the user's machine.

Sensitive credentials stored in the database (such as the Odoo API key) are Fernet-encrypted at the application layer before being written.

5. How information is transmitted (data in transit)

5.1 Between the app and its own backend — loopback only

The frontend communicates with the backend exclusively over the **local loopback interface** (<http://127.0.0.1:8000> and the <ws://127.0.0.1:8000> WebSocket).

This traffic **never leaves the machine** and is not exposed to the local network or the internet. There is therefore no TLS on this hop by design — there is no network path for a remote party to intercept it. Requests still carry the bearer token and are authenticated/authorized as described above.

5.2 To external services — HTTPS

When the app legitimately needs to reach an external service, it does so over **HTTPS/TLS**:

- **Google Workspace** (sign-in, Gmail, Drive, Calendar, Sheets) via Google's OAuth 2.0 and REST APIs.
- **AI providers** — the language model (Anthropic) and embedding provider (Voyage) used to answer questions and index content.

5.3 What is shared with AI providers — important

To generate answers, summaries, drafts, and document embeddings, relevant content you ask Little Gerry to work with (for example the text of emails, documents, or

attachments involved in a task) is transmitted to the configured **third-party AI providers** over HTTPS for processing. This is inherent to how the assistant works. Administrators and users should treat the choice of AI provider as part of the data-handling policy, since that provider processes the content sent to it. No AI-provider API keys are shipped in the code — they are supplied per install and stored in the OS keyring (§4.1).

6. Tamper-evident audit logging

Security-relevant events (for example login success/failure and account provisioning) are written to an **append-only audit log** with a **SHA-256 hash chain** ([backend/services/audit/logger.py](#)):

- Each record's hash is computed deterministically over its fields **plus the previous record's hash** (a blockchain-style linkage).
 - Any modification, insertion, or deletion of a past event breaks the chain and is detectable.
 - Login events also capture IP address and user-agent for forensic context ([backend/routers/auth.py](#)).
-

7. Application & update integrity

- **Windows** — the installer is **Authenticode code-signed** (**CN=Precisian Medical Instruments**); Windows verifies the signature before install.
 - **macOS** — the **.pkg** is **signed and notarized** through the release CI pipeline, so Gatekeeper trusts it.
 - **Auto-update** — updates are pulled from the project's official GitHub Releases over HTTPS. The macOS auto-updater authenticates to GitHub with a read-only token bundled into the signed package.
-

8. Secret & credential management summary

Secret	Stored in	Notes
JWT signing key	OS keyring	64-byte random, generated on first run
Fernet data-encryption key	OS keyring	Generated on first run
AI provider API keys	OS keyring	Supplied per install
Refresh tokens	Database	SHA-256 hash only — never the raw token
Passwords	Database	bcrypt hash with per-hash salt
Odoo API key	Database	Fernet-encrypted
Google OAuth client secret	Not committed to source; git-ignored	Fetches/placed per install

9. Known limitations & caveats

These are stated plainly so operators can make informed decisions:

1. **Local transport is plain HTTP over loopback.** This is safe because the backend is bound to `127.0.0.1` and unreachable off-machine, but it does mean another process running **as the same OS user** could in principle observe loopback traffic. Protecting the OS login is part of the security model.
2. **The Google OAuth token file is currently stored as plaintext JSON** on disk (`google_token.json`, written via `creds.to_json()` in `backend/services/google_service.py`). Unlike attachments, documents, and the Odoo key, it is **not** Fernet-encrypted in the active code path — even though an encrypted column exists in the schema. This token grants access to the connected Google account, so it inherits the protection of the OS user account and file-system permissions only. Wrapping this file in Fernet (to match the other credentials) is a recommended hardening step.
3. **Content is shared with third-party AI providers** to deliver assistant features (see §5.3). This is by design and should be reflected in any organizational data-handling policy.

4. **Machine-level trust.** Full-disk encryption (BitLocker / FileVault), a strong OS login, and keeping the OS user account uncompromised are assumptions of this model, since all local data and keyring secrets are protected by the OS user context.

10. Reporting a security issue

If you discover a security vulnerability, do **not** open a public issue. Contact the application owner directly (morganjkeane@pmi-llc.com) with details and reproduction steps so it can be triaged and patched in a coordinated release.